

Le Dockerfile : De Votre Code Python au Conteneur d'Excellence

Un guide pratique pour construire des images légères, rapides et sécurisées.

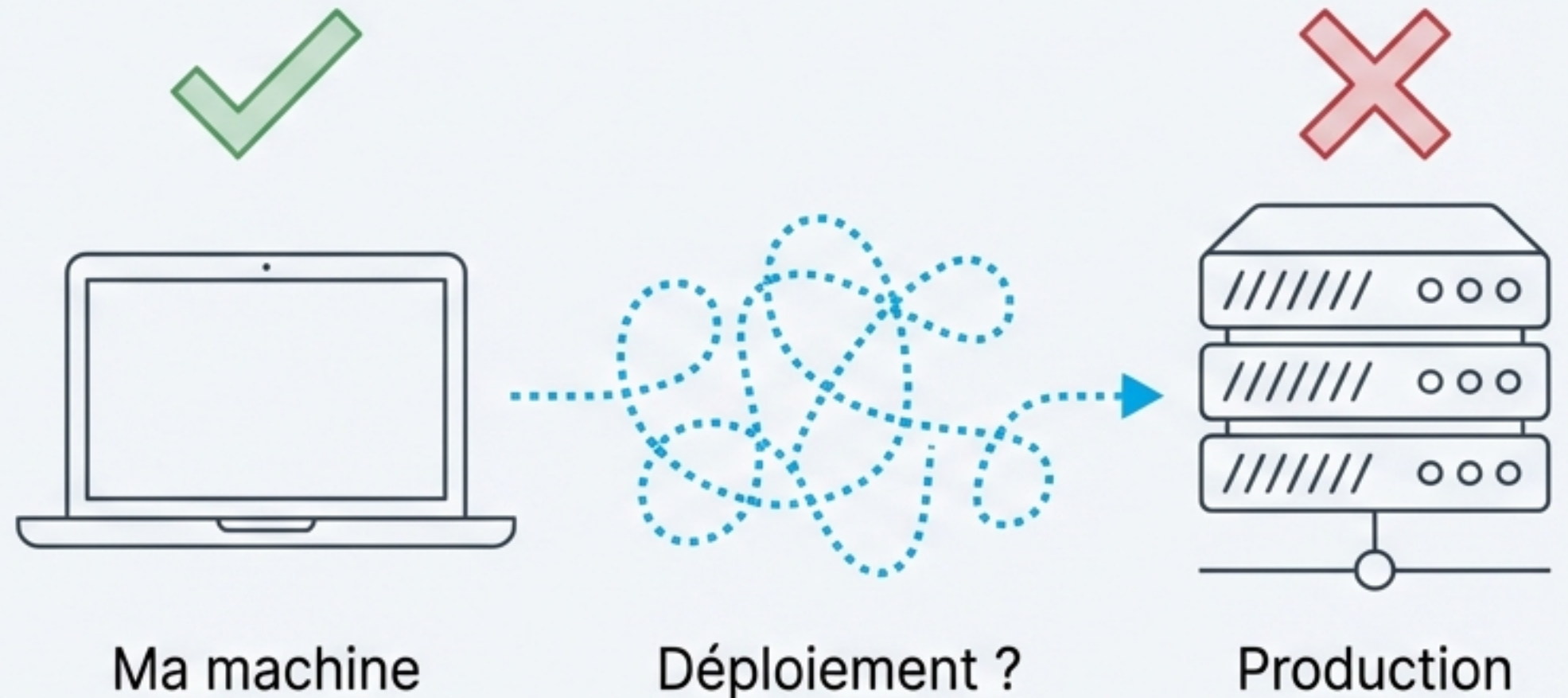


Le point de départ : « Ça marche sur ma machine ! »

Le scénario classique du développeur : une application Python qui fonctionne parfaitement en local, mais qui échoue lors du déploiement en production ou sur la machine d'un collègue.

Causes communes de ce problème :

- Conflits de versions (Python, bibliothèques)
- Différences d'environnement (variables, dépendances système)
- Complexité de la mise en place manuelle



La carte du monde Docker : Recette, Plat, et Portion



Dockerfile (La Recette)

Un fichier texte contenant la liste des instructions étape par étape pour construire l'environnement de votre application. C'est le plan.



Image (Le Plat Préparé)

Le résultat de l'exécution du Dockerfile. C'est un package immuable contenant votre code, vos dépendances et tout ce qui est nécessaire à son poire à son exécution. Il est prêt à être servi.



Conteneur (Une Portion Servie)

Une instance en cours d'exécution d'une image. C'est le plat en train d'être consommé. Vous pouvez lancer plusieurs conteneurs (portions) à partir de la même image (plat).

Pour ne pas confondre

- **Dockerfile** : Construit **UNE** image.
- **docker-compose** : Orchestre **PLUSIEURS** conteneurs pour faire fonctionner une application complète (ex: un conteneur pour l'API Python, un autre pour la base de données).

Anatomie d'un Dockerfile : Les 5 instructions vitales

Syntaxe : ``INSTRUCTION argument``

```
# Un Dockerfile doit commencer par FROM  
FROM python:3.11
```

FROM : Le point de départ. « Sur quelle base allons-nous construire ? » Spécifie l'image de base. Un Dockerfile **doit** commencer par ``FROM``.

```
WORKDIR /app
```

WORKDIR : Le plan de travail. « Où allons-nous travailler dans le conteneur ? » Définit le répertoire de travail pour les instructions suivantes.

```
COPY . /app
```

COPY : Ajouter les ingrédients. « Comment ajouter notre code ? » Copie des fichiers et des répertoires de l'hôte vers le conteneur.

```
RUN pip install flask
```

RUN : Les étapes de préparation. « Comment installer nos dépendances ? » Exécute des commandes dans une nouvelle couche de l'image. Utilisé pour installer des paquets.

```
CMD ["python", "app.py"]
```

CMD : La commande de service. « Comment lancer notre application ? » Spécifie la commande par défaut à exécuter au démarrage du conteneur.

Premier Essai : Dockerisons notre application Python (Flask/FastAPI)

Voici un premier jet pour notre application. Il fonctionne, mais nous allons voir qu'il est loin d'être optimisé.

app.py

```
# app.py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, Docker!'

if __name__ == '__main__':
    app.run(host='0.0.0.0')
```

Dockerfile V1

```
# Dockerfile V1 : Fonctionnel mais perfectible
FROM python:3.11

# 1. Définir le répertoire de travail
WORKDIR /app

# 2. Copier TOUT le contexte de build
COPY . .

# 3. Installer les dépendances
RUN pip install -r requirements.txt

# 4. Exposer le port de l'application
EXPOSE 5000

# 5. Lancer l'application
CMD ["python", "app.py"]
```


Le Diagnostic : Une image lourde, lente à builder et peu sécurisée



Problème 1 : Taille Excessive

L'image `python:3.11` est une image complète basée sur Debian. Elle contient des compilateurs et des outils de build (`gcc`, etc.) inutiles pour simplement *exécuter* notre application en production.

Conséquence : Des images de plusieurs centaines de Mo, un déploiement plus lent.



Problème 2 : Mauvaise Utilisation du Cache

L'instruction `COPY . .` est placée avant `RUN pip install`. Le moindre changement dans notre code (`app.py`) invalidera le cache de cette couche ET de toutes les suivantes, forçant la réinstallation de toutes les dépendances à chaque build.

Conséquence : Des temps de build inutilement longs en développement.



Problème 3 : Risque de Sécurité Majeur

Par défaut, toutes les instructions et le processus final sont exécutés avec l'utilisateur `root` à l'intérieur du conteneur.

Conséquence : Si un attaquant exploite une faille dans notre application, il obtient les privilèges `root` dans le conteneur, une porte d'entrée pour des attaques plus graves.

La Révolution : Le Build Multi-Étapes

Séparons la phase de *construction* (outils nécessaires) de la phase d'*exécution* (strict nécessaire).

Avant (Dockerfile V1)

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

Étape 1: Builder (python:3.11)



Code Source



Compilateur



Dépendances

`COPY --from=builder`

Bénéfice clé : Une image finale drastiquement plus petite, granthou, ne contenant veucine queie nécessaire a realiation pour lexécutior entrn lait en férottage.

Étape 2: Production (python:3.11-slim)

Après (Multi-Étapes)

```
# --- Étape 1: Le Builder ---
FROM python:3.11 AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt
```

```
# --- Étape 2: La Production ---
FROM python:3.11-slim
WORKDIR /app

COPY --from=builder /root/.local /root/.local
COPY . .

ENV PATH=/root/.local/bin:$PATH

EXPOSE 5000
CMD ["python", "app.py"]
```

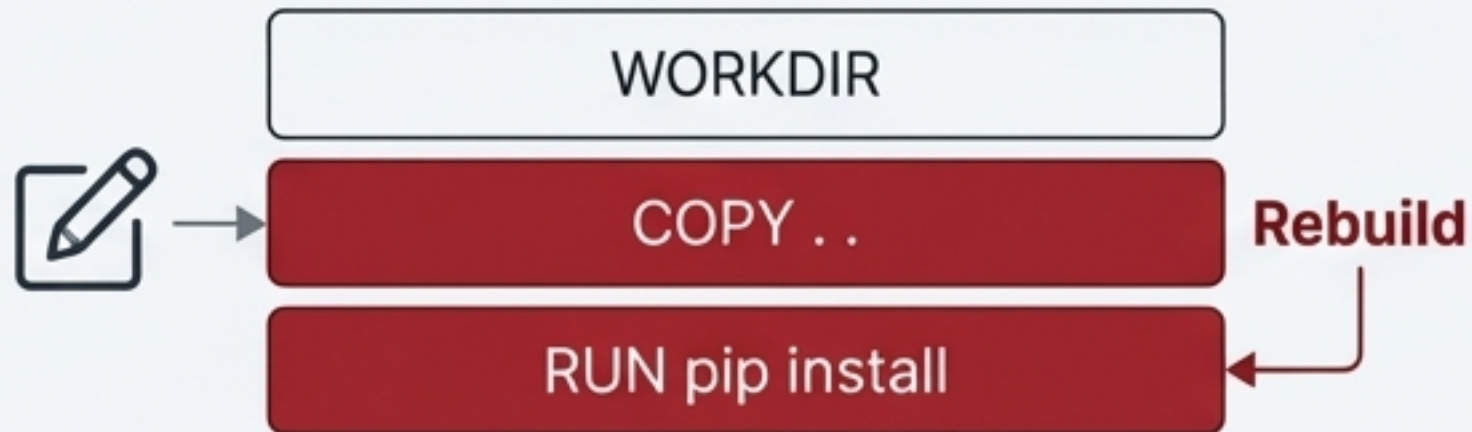
Bénéfice Clé : Une image finale drastiquement plus petite, ne contenant que le nécessaire pour l'exécution.

L'Art du Cache : Organisez vos couches intelligemment

Chaque instruction (`RUN`, `COPY`) crée une "couche". Si une couche change, Docker reconstruit cette couche et **toutes les suivantes**.

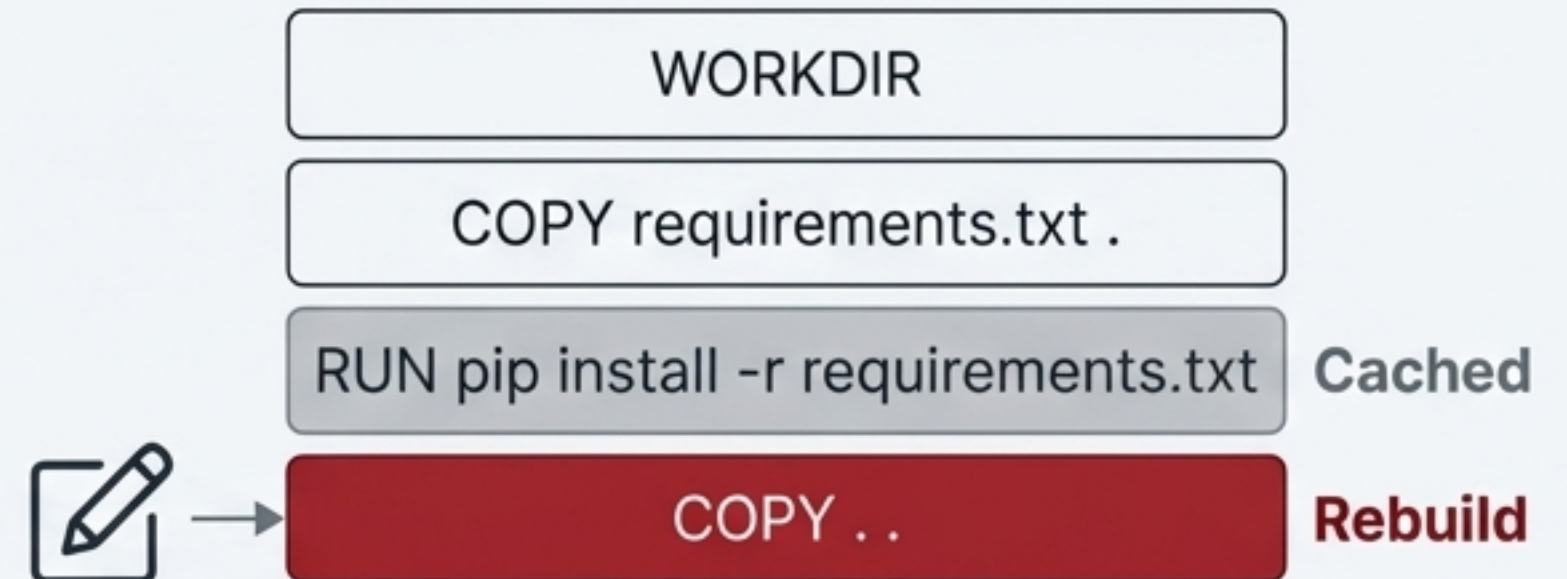
Mauvais

```
# Mauvais : COPY avant RUN pip install
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
```



Bon

```
# Bon : COPY requirements.txt d'abord
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

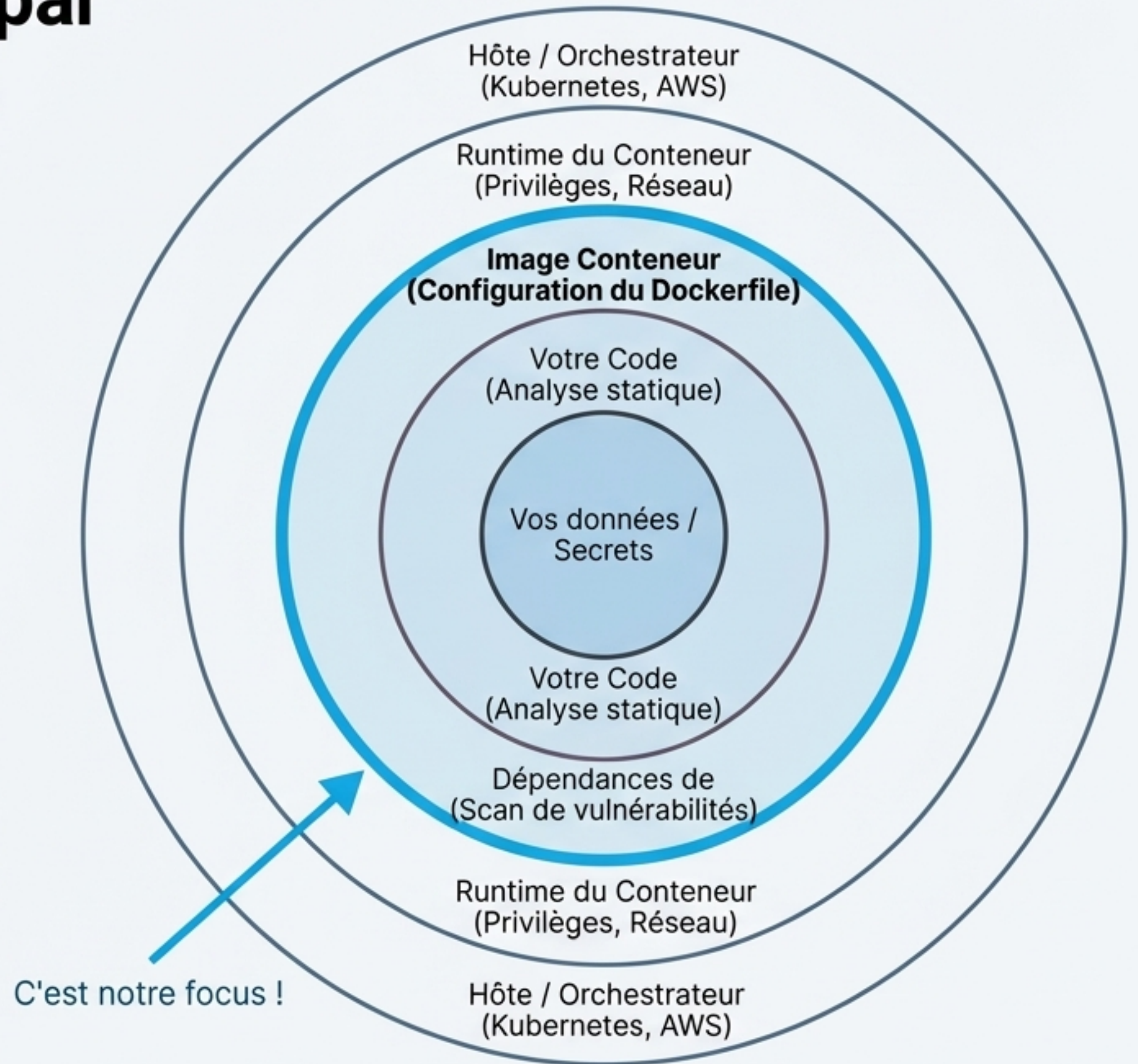


Résultat : Des builds quasi-instantanés lors des changements de code, car les dépendances n'ont pas besoin d'être réinstallées.

La Sécurité : Une approche par couches (Defense in Depth)

La sécurité d'un conteneur est une stratégie de défense en profondeur, comme les peaux d'un oignon. Le `Dockerfile` est la première couche de défense : il définit la composition et la configuration de base de notre image.

En appliquant les bonnes pratiques dans notre Dockerfile, nous créons une fondation solide pour la sécurité de toute l'application.



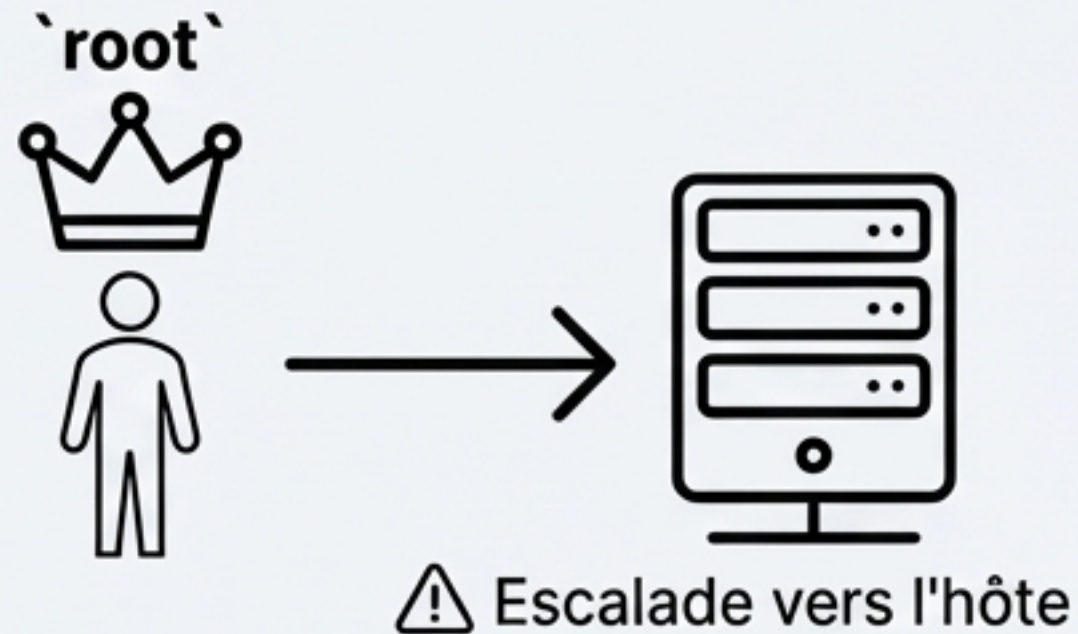
Pratique d'Excellence n°1 : Le principe du moindre privilège

Le Danger de `root`

Par défaut, un conteneur s'exécute en tant qu'utilisateur `root`. Si un attaquant compromet votre application, il obtient un accès `root` à l'intérieur du conteneur. Ceci facilite grandement les attaques d'escalade de privilèges vers l'hôte.

La Solution en 2 Étapes

1. **Créer un utilisateur dédié** : Utiliser `RUN` pour ajouter un utilisateur et un groupe système non-privilégiés.
2. **Activer cet utilisateur** : Utiliser l'instruction `USER` pour s'assurer que l'application s'exécute avec cet utilisateur.



```
# ...  
# Crée un groupe 'appgroup' et un utilisateur 'appuser'  
# -r : utilisateur système, -s /bin/false : pas de shell  
RUN groupadd -r appgroup && useradd -r -g appgroup -s /bin/false  
  
# ... (copie des fichiers, etc.)  
  
# S'assurer que l'utilisateur possède les fichiers de l'application  
RUN chown -R appuser:appgroup /app  
  
# Changer d'utilisateur pour le reste du build et pour l'exécution  
USER appuser  
  
# La commande CMD s'exécutera désormais en tant que 'appuser'  
CMD ["python", "app.py"]
```

Création de
l'utilisateur/groupe
système

Attribution
des permissions

Activation
de l'utilisateur

Pratique d'Excellence n°2 : Minimiser la surface d'attaque

Principe: Moins il y a de composants dans votre image, moins il y a de failles potentielles à exploiter.

Action 1 : Choisir la bonne image de base



- ``python:3.11``: Complète, idéale pour le build.
- ``python:3.11-slim``: Bon compromis pour la production.
- ``python:3.11-alpine``: Très légère, mais attention à la compatibilité (``musl``).
- ``distroless``: Extrême, ne contient que l'application et ses dépendances.

Action 2 : Ne pas installer d'outils inutiles

L'image de production ne devrait pas contenir d'outils comme ``curl``, ``wget``, ``vim``, ou ``git``. Ils ne sont pas nécessaires à l'exécution de l'application et peuvent être utilisés par un attaquant.



Pratique d'Excellence n°3 : Gérer les secrets et les vulnérabilités

Les Secrets

La Règle d'Or : **NE JAMAIS COPIER de secrets** (clés d'API, `.env`) dans une image Docker.



Les Bonnes Pratiques : Injecter les secrets au moment de l'exécution (runtime) via des variables d'environnement ou des volumes. Pour le build, utiliser `docker build --secret`.

Les Vulnérabilités

Le Problème : Votre image de base et vos dépendances (`requirements.txt`) peuvent contenir des vulnérabilités connues (CVEs).

La Solution : Scannez vos images, idéalement dans votre pipeline CI/CD.

```
$ snyk test --docker mon-app-python:latest  
  
× High severity vulnerability found in flask  
× Medium severity vulnerability found in glibc  
  
Found 2 vulnerabilities.
```

Selon Snyk, 44% des scans d'images Docker contiennent des vulnérabilités pour lesquelles une image de base plus récente et plus sûre est disponible.

Notre Dockerfile Idéal : Sécurisé, Léger et Efficace

En combinant le build multi-étapes, l'optimisation du cache et les meilleures pratiques de sécurité, voici à quoi ressemble notre Dockerfile final.

```
# --- Étape 1: BUILDER ---
# Utilise une image "slim" pour le build, suffisante pour installer les wheels.
FROM python:3.11-slim AS builder

WORKDIR /app

# Crée un utilisateur non-root pour la sécurité (sera copié plus tard).
RUN groupadd -r appgroup && useradd -r -g appgroup -s /bin/false appuser

# Copie et installe les dépendances SÉPARÉMENT pour le cache.
COPY requirements.txt .
# Utilise pip wheel pour pré-compiler, évite le besoin d'outils de build en prod.
RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt
```

```
# --- Étape 2: PRODUCTION ---
# L'image finale est aussi "slim" pour une taille minimale.
FROM python:3.11-slim

# Récupère la configuration de l'utilisateur créé dans l'étape builder.
COPY --from=builder /etc/passwd /etc/passwd
COPY --from=builder /etc/group /etc/group

WORKDIR /app

# Copie les wheels pré-compilés et les installe SANS cache.
COPY --from=builder /wheels /wheels
RUN pip install --no-cache /wheels/*

# Copie le code de l'application à la fin pour optimiser le cache.
COPY . .

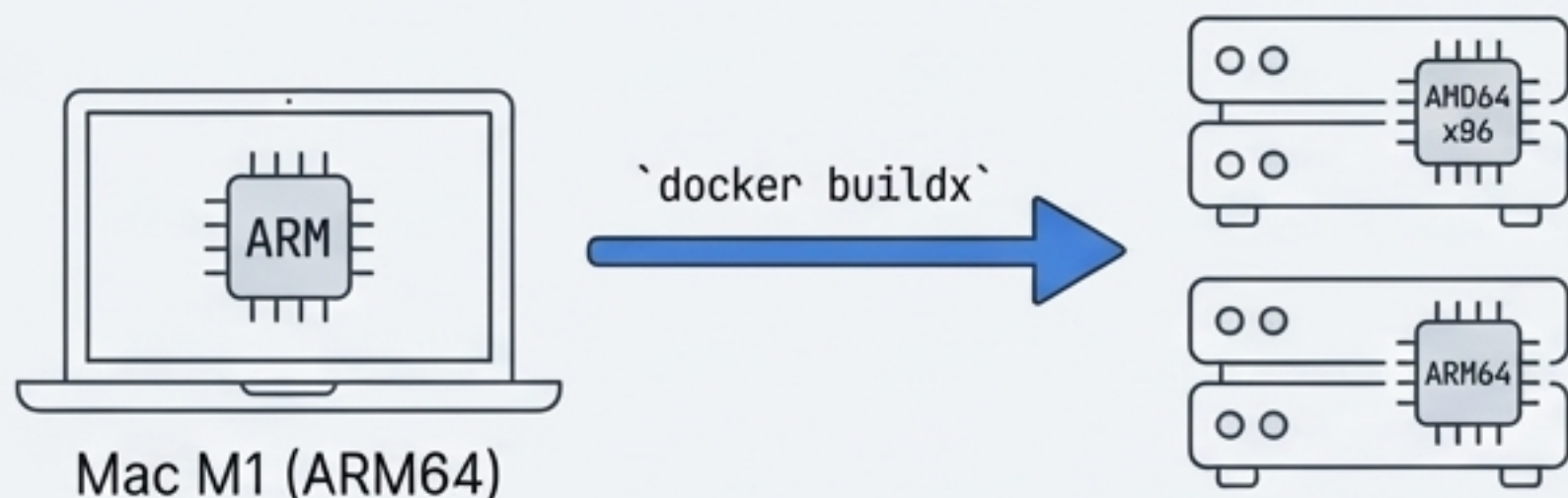
# L'application appartient et est exécutée par l'utilisateur non-root.
RUN chown -R appuser:appgroup /app
USER appuser

EXPOSE 5000
CMD ["python", "app.py"]
```


Aller plus loin : Builds multi-plateforme et Linting

Concept 1 : Builds Multi-Plateforme

Le besoin : Développer sur un Mac M1 (ARM64) et déployer sur un serveur Linux en production (AMD64).



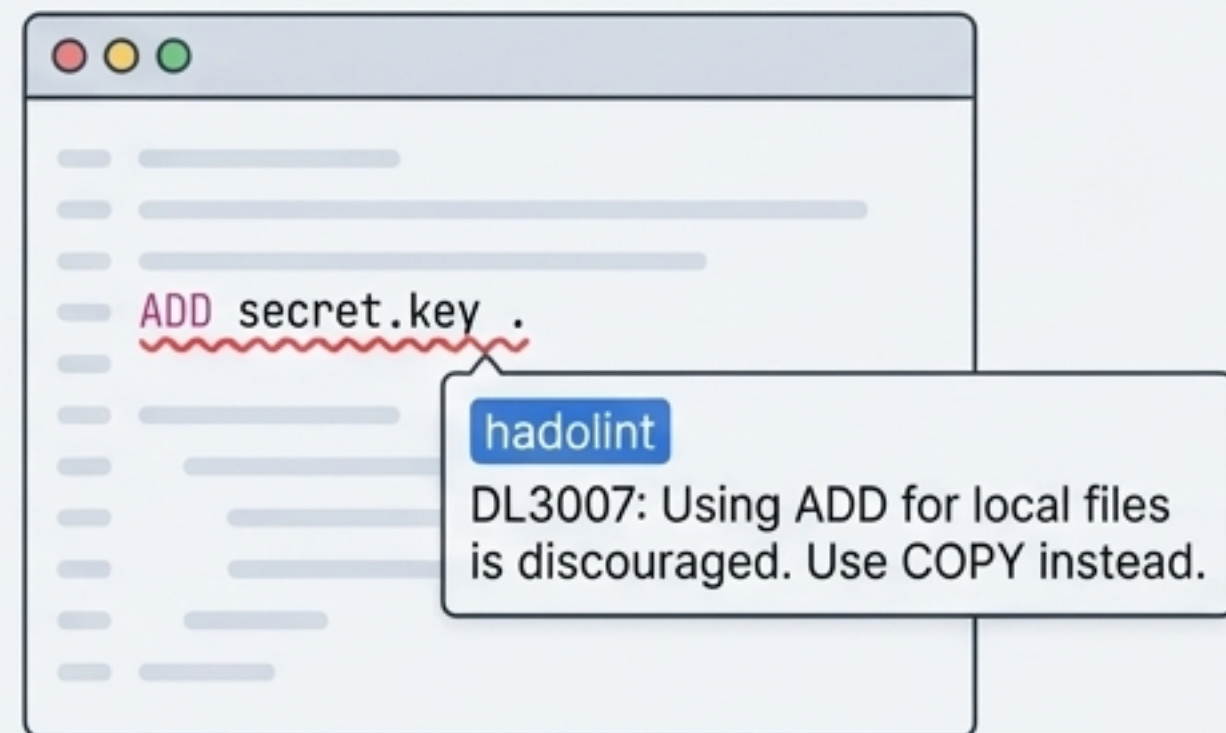
```
# Le build se fait toujours sur la plateforme native pour la vitesse
FROM --platform=$BUILDPLATFORM python:3.11-slim AS builder
...

ARG TARGETPLATFORM
# Des commandes spécifiques peuvent utiliser $TARGETPLATFORM
RUN GOARCH=$TARGETARCH go build ...
```

Concept 2 : Linting de Dockerfile

Le besoin : Automatiser la détection des erreurs et des mauvaises pratiques avant même de lancer le build.

L'outil : `hadolint`. Un linter qui analyse statiquement votre Dockerfile et vous avertit des problèmes.



Votre parcours de maîtrise du Dockerfile



1. CONSTRUIRE : Commencer avec une version simple et fonctionnelle.



2. OPTIMISER : Réduire la taille de l'image et accélérer les builds (Multi-étapes, cache).



3. SÉCURISER : Appliquer le moindre privilège, minimiser la surface d'attaque et scanner.

Les 3 réflexes du pro du Dockerfile



Penser 'minimalisme' : Quelle est l'image la plus petite possible pour mon besoin ?



Penser 'non-root' : Mon application doit-elle vraiment s'exécuter en tant que root ? (Non)



Penser "multi-étapes" : Puis-je séparer la construction de l'exécution ?

Maintenant, à vous de jouer ! Appliquez ces principes pour transformer les Dockerfiles de vos propres projets Python.